Comprehensive Guide: Tree Data Structures in Computer Science (DSA)

1. Introduction to Tree Data Structures & Terminology

A **Tree** is a hierarchical non-linear data structure consisting of nodes connected by edges. Unlike linear data structures (such as arrays, linked lists, stacks, and queues) which store data sequentially, trees store data in a parent-child relationship, making them ideal for representing hierarchical information like file systems, organization charts, and abstract syntax trees.

Core Terminology

- **Node:** The fundamental building block of a tree that contains data (and references/pointers to child nodes).
- **Root:** The topmost node in a tree. It has no parent. Every tree has exactly one root.
- **Edge:** The directed connection line linking a parent node to a child node.
- **Parent:** Any node that has one or more child nodes connected below it.
- **Child:** A node directly connected beneath another node (its parent).
- **Sibling:** Nodes that share the exact same parent node.
- **Leaf (External Node):** A node that does not have any children (i.e., its children pointers are null).
- **Internal Node:** Any node that has at least one child node.
- **Degree of a Node:** The total number of subtrees (children) attached to a node.
- **Path:** A sequence of consecutive nodes and edges connecting a source node to a destination node.
- **Depth of a Node:** The length of the path from the root node to that specific node (Root has depth 0).
- **Height of a Node:** The length of the longest path from that node down to a leaf node (Leaves have height 0; tree height equals root height).
- **Subtree:** A node and all of its descendants combined, forming a valid tree on its own.

---

2. Binary Trees & Binary Search Trees (BST)

Binary Tree

A **Binary Tree** is a specialized tree data structure where every node has **at most two children**, conventionally referred to as the **left child** and the **right child**.

Types of Binary Trees:

- **Full (or Strict) Binary Tree:** Every node other than the leaves has exactly two children.
- **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from as far left as possible.
- **Perfect Binary Tree:** All internal nodes have two children, and all leaf nodes are at the exact same depth/level.
- **Balanced Binary Tree:** The height of the left and right subtrees of any node differ by at most one (e.g., AVL Trees, Red-Black Trees).

Binary Search Tree (BST)

A **Binary Search Tree** is a node-based binary tree data structure that satisfies the following ordered property (BST Property):

1. The left subtree of a node contains only nodes with keys **less than** the node's key.
2. The right subtree of a node contains only nodes with keys **greater than** the node's key.
3. Both the left and right subtrees must also be valid binary search trees (no duplicate keys are typically allowed in a standard BST).

Core BST Operations:

- **Search ($O(\log n)$ average, $O(n)$ worst-case):** Recursively compare the target key with the current node. If smaller, go left; if larger, go right.
- **Insertion ($O(\log n)$ average, $O(n)$ worst-case):** Traverse down following the BST property until a null position is found, then create and attach the new node.
- **Deletion ($O(\log n)$ average, $O(n)$ worst-case):**
    - *Case 1 (Leaf Node):* Simply remove the node.
    - *Case 2 (Node with 1 Child):* Remove the node and link its parent directly to its child.
    - *Case 3 (Node with 2 Children):* Find the in-order successor (minimum in the right subtree) or in-order predecessor, swap values, and delete that successor node.

---

3. Tree Traversals

Tree traversal refers to the process of visiting every node in a tree exactly once. Unlike linear structures, trees can be traversed in multiple distinct orders.

A. Depth-First Search (DFS) Strategies

1. **Pre-order Traversal (`Root -> Left -> Right`):**
    - *Use Case:* Creating a copy of the tree, or prefix expression notation (Polish notation).
2. **In-order Traversal (`Left -> Root -> Right`):**
    - *Use Case:* In a Binary Search Tree, in-order traversal retrieves the keys in **ascending sorted order**.
3. **Post-order Traversal (`Left -> Right -> Root`):**
    - *Use Case:* Deleting or freeing a tree from bottom-up, evaluating postfix expressions (Reverse Polish notation).

B. Breadth-First Search (BFS)

1. **Level-order Traversal:**
    - Visits nodes level-by-level, starting from the root and moving from left to right across each level.
    - *Implementation:* Typically implemented iteratively using a **Queue** data structure.

---

4. Self-Balancing Binary Search Trees

Standard BSTs can degenerate into linear linked lists under sequential insertions (e.g., inserting sorted data 1, 2, 3, 4, 5), degrading time complexities from $O(\log n)$ to $O(n)$.
Self-balancing trees automatically maintain a balanced height during insertions and deletions via tree rotations.

- **AVL Trees:**
    - Named after Adelson-Velsky and Landis.
    - Strictly enforces that the balance factor (height difference between left and right subtrees) of any node is between `-1` and `+1`.
    - Uses single rotations (Left-Left, Right-Right) and double rotations (Left-Right, Right-Left) to restore balance.
- **Red-Black Trees:**

- Uses node color properties (Red or Black) and rules ensuring no path is more than twice as long as any other path.
- Provides slightly looser balancing than AVL trees, resulting in fewer rotations and faster insertion/deletion performance in practice (used extensively in Java's `TreeMap`/`TreeSet` and C++ STL `std::map`).

---

5. Advanced Trees & Real-World Applications

Specialized Tree Variants

- **Trie (Prefix Tree):** An ordered tree data structure used to store associative data streams, widely used in autocomplete systems, spell-checkers, and IP routing tables.
- **Heap (Min-Heap / Max-Heap):** A complete binary tree satisfying the heap property (parent is smaller/larger than children), fundamental for Priority Queues and Heap Sort algorithms.
- **Segment Tree / Fenwick Tree (Binary Indexed Tree):** Specialized trees used for efficient range query operations and range updates on arrays (e.g., Range Minimum/Sum Queries).

Real-World Applications in Computer Science

1. **File Systems:** Operating systems (like Linux and Windows) organize directories and files in tree structures.
2. **Databases:** Indexing mechanisms (such as **B-Trees** and **B+ Trees**) allow database management systems to perform fast disk-based record lookups.
3. **Compilers:** Abstract Syntax Trees (ASTs) represent the syntactic structure of source code during compilation.
4. **Networking & Routing:** Routing algorithms utilize spanning trees and decision trees to determine optimal packet paths.